

STUDENT PROJECT BOOK

Intermediate C++ Project Book

Extended Activities & Capstone Challenges — Part 2 Companion

Introduction to C++ Programming — Part 2: Intermediate C++

How to Use This Book

This book is a companion to Part 2 of the Introduction to C++ Programming course (Modules 10-18). It does not re-teach the lessons — read each module's README and complete its own exercise and project first. What follows here are extended, independent activities for each topic: more practice, applied to new scenarios, with no starter code and no solutions provided.

That's deliberate. By Part 2, you have enough tools to design a solution from a plain-language problem statement on your own — these activities are where that independence gets built. Attempt each one honestly before looking anything up; struggling productively with a problem you don't immediately see the answer to is the whole point.

Each chapter offers three activities of increasing difficulty, followed by an optional domain twist you can apply to any of them if you'd like extra practice applying the idea to Electrical/Electronic Engineering or Biomedical Engineering scenarios. The final chapter is an Extended Capstone — a bigger, optional challenge that goes beyond Module 19's own capstone by combining more of the course into one system.

Commit your work for each activity to your course GitHub repository, in a folder such as `project-book/module-10/`, exactly as you've been committing every exercise and project so far. This book is meant to grow your portfolio, not just your practice.

Chapter 10 — Pointers & Dynamic Memory

Recap: A pointer stores an address; & reads an address off a variable, * follows one back to a value. new[]/delete[] let you allocate an array whose size is only known at runtime — and every new[] needs exactly one matching delete[].

Activity 1 — Swap Without Copying

DIFFICULTY: **FOUNDATIONAL**

Scenario: You have two dynamically allocated arrays of the same size, each full of data. Copying every element from one to the other to "swap" them would be wasteful for large arrays.

Requirements:

- Write a function void swapArrays(double*& a, double*& b) that swaps which array each pointer refers to, without touching a single element inside either array.
- Note the double*& — a reference to a pointer. Explain in a comment why a plain double* parameter (without the reference) would NOT actually swap anything the caller sees.
- Demonstrate in main with two small arrays, printing their contents before and after the swap to prove the arrays themselves swapped, not their contents.

Activity 2 — A Growable Buffer

DIFFICULTY: **CORE**

Scenario: std::vector grows automatically. Build the mechanism that makes that possible, by hand, once, so you understand what it's hiding.

Requirements:

- Write a function double* resizeBuffer(double* oldBuffer, int oldSize, int newSize) that: allocates a new array of newSize, copies every element from oldBuffer into it (up to the smaller of the two sizes), deletes oldBuffer, and returns the new pointer.
- Demonstrate growing a small buffer at least twice in main, printing its contents after each resize to confirm old data survived.
- Explain in a comment: what would go wrong if you deleted oldBuffer BEFORE copying its contents into the new one?

Activity 3 — A Fixed-Capacity Ring Buffer

DIFFICULTY: **CHALLENGE**

Scenario: Sometimes you only care about the most recent N readings, and want old ones automatically discarded rather than growing forever.

Requirements:

- Build a small class RingBuffer with a dynamically allocated array of a fixed capacity (set in the constructor), that overwrites the OLDEST value once full instead of growing.
- Support add(double value) and a way to print the buffer's current contents in the order they were added.

- Free the array in a destructor. No leaks, no dangling pointers.

Domain Twist (optional)

EE twist: frame the ring buffer as an oscilloscope's rolling sample window. Biomedical twist: frame it as a patient monitor's rolling window of the last N heart-rate readings, used to compute a short-term rolling average.

Chapter 11 — Inheritance

Recap: Inheritance models an "is-a" relationship: a derived class gets everything a base class has, for free, and adds or overrides what's different. `protected` exposes base members to derived classes without exposing them to the outside world. A derived constructor must explicitly construct its base part first.

Activity 1 — Complete the Shape Family

DIFFICULTY: **FOUNDATIONAL**

Scenario: The course's own Shape hierarchy (Module 11's project) only covers Circle and Rectangle.

Requirements:

- Add a Triangle derived class with base and height, and its own `area()` calculation.
- Add a Square derived class — but derive it from Rectangle, not Shape directly (a square IS a rectangle with equal sides). Its constructor should take only one side length.
- In main, create one of each of the four shapes and print each one's name and area.

Activity 2 — Payroll

DIFFICULTY: **CORE**

Scenario: A company pays some employees a fixed salary and others by the hour.

Requirements:

- Build a base class Employee with a protected name and a virtual (yes, jump slightly ahead to Module 12's keyword here, or leave it non-virtual and just note the limitation in a comment) `double calculatePay()`.
- Add `SalariedEmployee` (a fixed monthly amount) and `HourlyEmployee` (hours worked times an hourly rate, with 1.5x pay for any hours over 40) as derived classes, each with its own `calculatePay()`.
- In main, create one of each, calling `calculatePay()` through each object's own (non-base) variable, and print the results.

Activity 3 — A Three-Level Hierarchy

DIFFICULTY: **CHALLENGE**

Scenario: Inheritance can chain: a class can derive from a class that itself derived from something else.

Requirements:

- Build Vehicle (make, topSpeed) -> Car : public Vehicle (adds numDoors) -> SportsCar : public Car (adds a bool hasTurbo).
- Every level's constructor must correctly call the one above it.
- Print a SportsCar object's full description, showing fields from all three levels appear correctly.

Domain Twist (optional)

EE twist: build Instrument -> Multimeter -> AutoRangingMultimeter. *Biomedical twist:* build MedicalDevice -> VitalSignMonitor -> ICUMonitor (adding an alarm threshold field). Both are direct three-level extensions of Activity 3.

Chapter 12 — Polymorphism & Virtual Functions

Recap: virtual makes a base class method dispatch to the actual object's type at runtime, but only through a pointer or reference — passing by value still slices. override is a compile-time safety check. A polymorphic base class needs a virtual destructor.

Activity 1 — A Polymorphic Zoo

DIFFICULTY: **FOUNDATIONAL**

Scenario: A zoo has many kinds of animals, and a keeper wants one feeding report for all of them.

Requirements:

- Build a base class Animal with a virtual std::string makeSound() and a virtual void eat(), plus a virtual destructor.
- Add at least three derived classes (e.g. Lion, Parrot, Snake), each overriding both methods.
- Build a std::vector<Animal*>, populate it with new objects of each type, and print one feeding report with a single polymorphic loop.
- delete every pointer before the program ends.

Activity 2 — Make Chapter 11's Hierarchy Polymorphic

DIFFICULTY: **CORE**

Scenario: Your Shape family from Chapter 11 currently can't be handled as a mixed collection.

Requirements:

- Go back to your Triangle/Square/Circle/Rectangle hierarchy. Add virtual to area() on the base Shape and virtual ~Shape().
- Build a std::vector<Shape*> containing at least one of every shape type.
- Print each shape's area with one polymorphic loop, and print the TOTAL area of the whole collection.

Activity 3 — A Slicing Bug Hunt

DIFFICULTY: **CHALLENGE**

Scenario: Someone on your team wrote code that compiles fine but silently gives wrong output.

Requirements:

- Deliberately write a small program with a polymorphic hierarchy where ONE function accidentally takes a base-class parameter by value instead of by reference (recreate the slicing bug on purpose).
- Add a code comment explaining exactly why it's wrong, and then fix it.
- Write a short paragraph (as a comment block) describing how you would explain this bug to a teammate who didn't take this course.

Domain Twist (optional)

EE twist: extend Chapter 11's Instrument hierarchy into a vector<Instrument*> "equipment rack" with one polymorphic calibrate() report. *Biomedical twist:* extend the MedicalDevice hierarchy into a polymorphic "code cart" report listing every device's status in an emergency scenario.

Chapter 13 — Operator Overloading

Recap: Overload operator+ and operator== as const methods taking a const reference; overload operator<< as a friend free function, since the left-hand side of << is a stream you don't own. Only overload an operator when its built-in meaning genuinely transfers to your type.

Activity 1 — A Money Class

DIFFICULTY: **FOUNDATIONAL**

Scenario: Storing money as a plain double invites rounding errors. Store it as whole cents internally instead.

Requirements:

- Build a class Money with a private long cents, and a constructor taking dollars as a double (converting to cents internally, rounded).
- Overload +, -, ==, and < to combine/compare Money values correctly using the underlying cents.
- Overload << as a friend function to print as "\$12.50" (two decimal places, computed from cents).

Activity 2 — A 2x2 Matrix

DIFFICULTY: **CORE**

Scenario: Small fixed-size matrices show up constantly in engineering and graphics work.

Requirements:

- Build a class Matrix2x2 storing four doubles (e.g. a, b, c, d for a 2x2 grid).
- Overload operator+ (element-wise addition) and operator* (real 2x2 matrix multiplication, not element-wise).
- Overload << to print the matrix on two lines, nicely aligned.

Activity 3 — Operator Overloading Judgment Review

DIFFICULTY: **CHALLENGE**

Scenario: Not every operator overload is a good idea, even if it compiles.

Requirements:

- Design a class where you INTENTIONALLY overload an operator badly (its meaning doesn't match the built-in operator's conventional meaning).
- Write a short comment explaining specifically why a reader would be misled, and what a better-named method would look like instead.
- Then write the GOOD version: replace the misleading operator overload with a clearly named method that does the same thing.

Domain Twist (optional)

EE twist: build a Temperature class in Celsius, overloading +, <, and << with a comment explaining why overloading + to combine two ABSOLUTE temperatures (rather than a temperature and a delta) is actually the kind of questionable overload Activity 3 asks you to critique. *Biomedical twist:* apply the same critique to a BloodPressure class storing systolic/diastolic — is + even meaningful for two blood pressure readings?

Chapter 14 — File I/O

Recap: `std::ofstream/std::ifstream` reuse `cout/cin`'s `<<` and `>>` syntax. Always check if `!file` before trusting a stream. `std::ios::app` appends instead of overwriting. `getline` + `stringstream` splits a CSV line into fields.

Activity 1 — A Persistent Contact Book

DIFFICULTY: **FOUNDATIONAL**

Scenario: A simple address book that remembers contacts between runs.

Requirements:

- Build a menu (Add Contact, Search by Name, List All, Exit) backed by a CSV file (name,phone), read fully into memory at startup and re-written completely on Add (not appended, since a duplicate name should update rather than duplicate).
- Handle the case where the file doesn't exist yet on the very first run.
- Search should be case-insensitive.

Activity 2 — A Sorted High Score Table

DIFFICULTY: **CORE**

Scenario: A game wants to keep a persistent, always-sorted top-10 leaderboard.

Requirements:

- Read an existing `scores.csv` (name,score) into a vector of structs at startup (create the file if missing).
- After adding a new score, re-sort descending by score, keep only the top 10, and rewrite the whole file (not appended).
- Print the leaderboard, ranked, after every new entry.

Activity 3 — A Lab Notebook With a Summary Report

DIFFICULTY: **CHALLENGE**

Scenario: A researcher wants to append timestamped notes all week, then generate one summary report on demand.

Requirements:

- Build a menu: Add Note (appends "timestamp,note text" to a log file, `std::ios::app`), and Generate Report.
- Generate Report should re-read the ENTIRE file and print: total note count, and every note whose text contains a keyword the user provides at report time.
- Handle a note that itself contains a comma without breaking your CSV parsing (hint: consider a different delimiter, or note this limitation explicitly in a comment as a known constraint).

Domain Twist (optional)

EE twist: turn Activity 3 into a "calibration log" for a piece of test equipment. *Biomedical twist:* turn it into a "patient visit log" for a single patient across many appointments — in both cases, the report step is the same skill, just renamed.

Chapter 15 — Exception Handling

Recap: throw signals a problem; try/catch responds without crashing. Catch specific exception types before a general `std::exception` safety net. Validate a class's invariant inside its constructor so an invalid object can never exist.

Activity 1 — An ATM Withdrawal Validator

DIFFICULTY: **FOUNDATIONAL**

Scenario: An ATM must reject several different kinds of bad withdrawal requests, each for a different reason.

Requirements:

- Write a function `double withdraw(double balance, double amount, double dailyLimit)` that throws `std::invalid_argument` for a negative/zero amount, a custom or `std::runtime_error` for `amount > balance` (insufficient funds), and another for `amount > dailyLimit`.
- In main, loop reading withdrawal attempts, catching each exception type separately with a distinct, specific message.
- Track and print how many attempts succeeded vs. were rejected, and why.

Activity 2 — A Custom Exception Type

DIFFICULTY: **CORE**

Scenario: The standard exception types are generic. Sometimes a more specific, named exception makes calling code clearer.

Requirements:

- Define your own exception class (e.g. `class NegativeValueError : public std::invalid_argument`) with a constructor that builds a helpful message.
- Use it inside a small class's constructor or setter to reject an invalid value.
- Catch it specifically in main (not just as `std::exception`) and explain in a comment why a specific catch is more useful here than a generic one.

Activity 3 — Exception Safety Under Load

DIFFICULTY: **CHALLENGE**

Scenario: A batch process reads 20 records from a vector of raw strings, some deliberately malformed.

Requirements:

- Given a `std::vector<std::string>` of "records" (some valid numbers, some garbage text), loop through all of them using `std::stod` inside a try/catch PER RECORD, so one bad record doesn't stop the whole batch.
- Print a final summary: how many parsed successfully, how many failed, and the average of the successful values.
- This is deliberately similar to Module 14's CSV work — note in a comment which Module 14 idiom this activity is combining with exception handling.

Domain Twist (optional)

EE twist: reframe Activity 1 as a circuit safety checker rejecting overcurrent/overvoltage conditions with distinct exception types per violation. Biomedical twist: reframe it as a dosage calculator rejecting a negative weight, a zero/negative rate, and a dose exceeding a safety ceiling — three distinct throw sites, three distinct catches.

Chapter 16 — STL: Containers & Iterators

Recap: `std::map` looks values up by key, not position, and stays sorted by key automatically. `map[key]` silently creates a missing key — use `.count()/find()` to check safely. `std::set` enforces uniqueness. A range-based for loop is an iterator loop the compiler writes for you.

Activity 1 — Word Frequency Counter

DIFFICULTY: **FOUNDATIONAL**

Scenario: Given a paragraph of text, find out which words appear most often.

Requirements:

- Read a sentence or short paragraph from the user (`getline`).
- Split it into words (a simple stringstream split on spaces is fine — don't worry about punctuation removal unless you want the extra challenge).
- Count occurrences with a `std::map<std::string, int>`, then print every word and its count, sorted by word (free, since map sorts by key).

Activity 2 — Unique Visitor Tracker

DIFFICULTY: **CORE**

Scenario: A building logs every entry badge swipe, including repeats from the same person all day.

Requirements:

- Given a `vector<std::string>` of badge IDs representing a day's swipes (with intentional repeats), use a `std::set<std::string>` to determine how many UNIQUE people entered.
- Also use a `std::map<std::string, int>` to count how many times EACH person swiped, and print anyone who swiped more than 3 times as "frequent visitor."

Activity 3 — A Small Registry Class

DIFFICULTY: **CHALLENGE**

Scenario: Wrap a map in a small class instead of using it directly, so the rest of your program can't misuse it.

Requirements:

- Build a class `Registry` that privately holds a `std::map<std::string, double>` and exposes: `void set(std::string key, double value)`, `bool has(std::string key) const` (using `.count()`, never `[]`), `double get(std::string key) const` (throwing `std::out_of_range` — Module 15 callback — if the key is missing), and `void printAll() const`.
- This is encapsulation (Module 8) applied directly to an STL container — explain in a comment why hiding the raw map behind `get/set/has` is safer than exposing it publicly.

Domain Twist (optional)

EE twist: apply Activity 3's Registry pattern to component values keyed by designator ("R1", "C1", ...).

Biomedical twist: apply it to a patient's latest vital signs keyed by vital-sign name ("heartRate", "spo2", ...)

— both are the identical class, renamed.

Chapter 17 — STL Algorithms & Lambdas

Recap: `std::sort`/`max_element`/`min_element`/`find` all operate on a `[begin, end)` range. A lambda is a function written inline: `[](params){ body }`. `count_if` uses a lambda predicate to count matches; `transform` builds a new sequence by applying a lambda to every element. Capture a variable by naming it: `[threshold]`.

Activity 1 — Custom Sort With a Comparator

DIFFICULTY: **FOUNDATIONAL**

Scenario: `std::sort`'s default order (ascending, using `<`) isn't always what you want.

Requirements:

- Given a struct `Player { std::string name; int score; }` and a `vector<Player>`, use `std::sort` with a lambda COMPARATOR (a lambda taking two `Players` and returning `bool`) to sort by score, HIGHEST first.
- Print the sorted leaderboard with rank numbers (1st, 2nd, 3rd, ...).
- This lambda takes two parameters instead of one — note in a comment how it differs in shape from every `count_if`/`transform` lambda you've written so far.

Activity 2 — The Erase-Remove Idiom

DIFFICULTY: **CORE**

Scenario: Module 16 showed erasing one element at a time with a loop. There's a faster, more idiomatic way to remove ALL elements matching a condition at once.

Requirements:

- Given a `vector<double>` of sensor readings including some obviously invalid values (e.g. negative, when negative is impossible for this sensor), use `std::remove_if` with a lambda to move all invalid values to the end, then `vector::erase` to actually remove them — the standard "erase-remove idiom."
- Print the vector before and after cleaning.
- Research (briefly, in a comment) why `remove_if` alone doesn't shrink the vector, and `erase` is still needed afterward.

Activity 3 — A Two-Stage Analysis Pipeline

DIFFICULTY: **CHALLENGE**

Scenario: Real analysis is rarely one algorithm — it's usually several chained together.

Requirements:

- Starting from a raw `vector<int>` of sensor counts, chain: (1) `transform` to convert to a calibrated unit using a lambda, (2) `remove_if` + `erase` to drop out-of-range calibrated values, (3) sort the cleaned data, (4) compute and print the mean of what's left (`max_element`/`min_element` optional extra).
- Structure this as a sequence of clearly commented steps, not one dense line — readability matters as much as correctness here.

Domain Twist (optional)

EE twist: Activity 3 as a waveform-cleaning pipeline (raw ADC -> voltage -> drop clipped/saturated samples -> sorted -> mean). Biomedical twist: the same pipeline as a vitals-cleaning pipeline (raw sensor -> clinical units -> drop sensor-fault readings -> sorted -> mean) — identical shape, different units and thresholds.

Chapter 18 — Templates

Recap: template `<typename T>` lets you write a function or class once and have the compiler generate a real version per type actually used — the same mechanism behind `std::vector<T>` and `std::map<K,V>` themselves. `T` must support whatever operations the template body actually uses.

Activity 1 — A Generic Stack

DIFFICULTY: **FOUNDATIONAL**

Scenario: A stack (last-in, first-out) is one of the most common building blocks in programming.

Requirements:

- Build a class template `Stack<T>` wrapping a `std::vector<T>` internally, exposing `push(T value)`, `pop()` (removes and returns the top), `top() const` (peeks without removing), and `isEmpty() const`.
- Have `pop()/top()` throw `std::runtime_error` if called on an empty stack (Module 15 callback).
- Demonstrate with a `Stack<int>` and a `Stack<std::string>` in `main`.

Activity 2 — A Generic Swap Function

DIFFICULTY: **CORE**

Scenario: Module 7 taught pass-by-reference swapping for one type at a time.

Requirements:

- Write template `<typename T> void swapValues(T& a, T& b)` that works for ANY type supporting copy (`int`, `double`, `std::string`, and your own classes).
- Demonstrate it swapping two `ints`, two `doubles`, and two instances of a small class you define (e.g. a `Point` with `x, y`).
- This is the templated generalization of Module 7's exercise — note in a comment exactly what changed and what stayed the same.

Activity 3 — A Templated Safe Divide

DIFFICULTY: **CHALLENGE**

Scenario: Division by zero is a problem regardless of whether you're dividing `ints` or `doubles`.

Requirements:

- Write template `<typename T> T safeDivide(T a, T b)` that throws `std::invalid_argument` if `b == 0` (using the type's own default-constructed zero, `T()`, for the comparison so it works generically), otherwise returns `a / b`.
- Demonstrate it with both `int` and `double` calls, including at least one divide-by-zero for each type, caught and handled.
- This combines Module 15 (exceptions) and Module 18 (templates) in one function — the exact kind of combination the Module 19 capstone asks you to do at a larger scale.

Domain Twist (optional)

EE twist: build `Statistics<T>` (mean, min, max) and feed it a `Stack<double>` of recent voltage readings via `repeated pop()`. Biomedical twist: feed the same `Statistics<T>` a `Stack<double>` of recent heart-rate readings — reuse, don't rebuild, the `Statistics<T>` class from Module 18's own project if you already built one.

Extended Capstone — The Complete System

Module 19's own capstone already combines every Part 2 concept into one staged, track-based project. This Extended Capstone is optional and goes further: instead of ONE class hierarchy, you'll build TWO that interact, add a written design-reflection component, and push your STL/template reuse across both hierarchies. Treat it as a portfolio centerpiece, not a required assignment — attempt it once you're genuinely comfortable with Module 19's own capstone.

Pick the same track you used for Module 19 (Generic, EE, or Biomedical) and extend that exact project. All three tracks share the same structural requirements below, applied to their own domain.

1. A second, interacting hierarchy

Add a SECOND polymorphic class hierarchy that interacts with your first one. Generic: add an Instructor hierarchy (Professor, TeachingAssistant) that can be ASSIGNED to Students. EE: add a Measurement hierarchy (VoltageMeasurement, CurrentMeasurement) that a Component can RECORD over time, distinct from the Component hierarchy itself. Biomedical: add a Staff hierarchy (Nurse, Physician) that can be ASSIGNED to a patient's MedicalDevice readings for sign-off.

2. Cross-hierarchy file persistence

Persist BOTH hierarchies to file, and — this is the new part — persist the RELATIONSHIP between them too (which instructor is assigned to which student; which staff member signed off which reading). Reloading at startup must reconstruct both hierarchies AND their relationships correctly.

3. Shared template reuse

Use the SAME Statistics<T> (or equivalent) template class against data from BOTH hierarchies — for example, statistics on student averages AND on instructor course-loads; or statistics on component impedances AND on measurement histories. Reusing one generic tool across unrelated data is the point.

4. A reporting feature using STL algorithms

Add one non-trivial report combining at least two STL algorithms (e.g. sort + count_if, or transform + max_element) that answers a genuine question about your combined system — e.g. "which instructor's students have the highest average?" or "which component's measurement history is the noisiest?"

5. A written design reflection

Add a REFLECTION.md file (not code) to your project answering, in your own words, in 200-400 words total: What was the hardest design decision in connecting your two hierarchies? What would you do differently if you were starting over? Which Part 1 AND Part 2 concept from the WHOLE course did you find most valuable, and why?

When You're Done

This project is intentionally open-ended — there is no single correct design for how two hierarchies should interact. Grade yourself (or ask your instructor to) against whether the system is exception-safe, genuinely persists across runs, avoids memory leaks, and whether your REFLECTION.md shows real, specific thinking

rather than generic statements. Commit it as its own repository folder, e.g. project-book/extended-capstone/, with its own staged commits exactly like every capstone before it.